

CS-1204

Object-Oriented Programming

(3+1)

By
Engr. Muhammad Awais

Contents

- Composition
- Aggregation

Aggregation

- Aggregation is a **has-a relationship** where one class contains another class, but the contained object can exist independently of the container.
- Lifespan: The **contained object is not destroyed** when the container is destroyed.
- Weak Relationship
- Whole-part relationship
- Loose Coupling
- Team ◇— Player

Aggregation Example

```
#include <iostream>
#include <string>
using namespace std;

class Player {
    string name;
public:
    Player(string n) {
        name = n;
    }

    void play() {
        cout << name << " is playing.\n";
    }

    ~Player() {
        cout << name << " is leaving the field.\n";
    }
};
```

```
class Team {
    Player* player; // Aggregation: Team has a Player, but doesn't own it
public:
    Team(Player* p) {
        player = p;
    }

    void startMatch() {
        cout << "Match started.\n";
        player->play();
    }
};
```

// No destructor needed to delete player -  owned!

```
int main() {
    Player* p1 = new Player("Alice"); // Player created independently
    {
        Team t1(p1); // Team aggregates existing player
        t1.startMatch();
        // Team goes out of scope and is destroyed
        // But Player is still alive
    }

    p1->play(); // Still accessible after Team is gone
    delete p1; // Now we manually destroy Player

    return 0;
}
```

Key Points

- Team does not delete Player.
- The player is created outside the Car. It exists independently.
- Player can exist before and after the Team object.
- Shows weak ownership and independent lifetimes—hallmark of Aggregation.

Composition

- Composition is a **strong has-a relationship** where the container class owns the contained object.
- Lifespan: The **contained object is destroyed** when the container is destroyed.
- Whole-part relationship
- Strong Relationship
- Tightly Coupled
- Car ◆—— Engine

Composition Example

```
class Engine {
    string engineNumber;
public:
    Engine() {
        engineNumber = "UNKNOWN";
        cout << "Default Engine created.\n";
    }

    void setEngineNumber(string number) {
        engineNumber = number;
    }

    void start() {
        cout << "Engine " << engineNumber << " starts.\n";
    }

    ~Engine() {
        cout << "Engine " << engineNumber << " destroyed.\n";
    }
};
```

```
class Car {
    Engine engine; // Composition: Car owns Engine
public:
    Car(string engineNum) {
        engine.setEngineNumber(engineNum); // Initialized inside constructor body
        cout << "Car created with engine " << engineNum << ".\n";
    }

    void startCar() {
        engine.start();
        cout << "Car is starting.\n";
    }

    ~Car() {
        cout << "Car destroyed.\n";
    }
};
```

```
int main() {
    {
        Car myCar("ENG-9876"); // Car and Engine constructed together
        myCar.startCar();
        // When Car goes out of scope, Engine is destroyed too
    }

    return 0;
}
```

Key Points

- Car and engine has strong relationship.
- Car class owns the engine.
- Engine can not exist independently.
- Engine object is destroyed when the car is destroyed.

Comparison

Feature	Aggregation	Composition
Ownership	No (shared object)	Yes (exclusive ownership)
Lifetime	Independent	Dependent
Deletion impact	Deleting container has no effect	Deleting container destroys parts
Reusability	Higher (same object can be reused)	Lower (tightly coupled)
Implementation	Pointer or reference	Direct member (object)

Quiz -> CLO 1

You are building a hospital system where:

- A Doctor can work in one or more Departments.
- A Patient has a MedicalRecord that includes diagnosis and prescriptions.
- A Hospital has several Departments.

Identify the relationship (Composition or Aggregation) that exist between the following classes. Also write one line reason for each.

- Doctor and Department?
- Patient and MedicalRecord?
- Department and Hospital?

Quiz Solution

Relationship	Type	Reason
Doctor and Department	Aggregation	A doctor can work in multiple departments, and departments can exist independently of any specific doctor. Hence, it's a <i>shared</i> relationship.
Patient and MedicalRecord	Composition	A medical record cannot exist without the patient; it is an essential part of the patient's data. If the patient is deleted, the medical record is also deleted.
Department and Hospital	Composition	Departments are integral parts of a hospital. If the hospital ceases to exist, its departments also cease to exist.

✓ Summary:

- Doctor–Department → Aggregation
- Patient–MedicalRecord → Composition
- Department–Hospital → Composition

Composition Task

Design a simple **Calculator** class that can perform basic arithmetic operations such as addition and multiplication.

The calculator should display results using a separate class called **Display**.

The **Calculator class must own a Display object** as a part of its internal structure.

When the calculator performs a calculation, it should delegate the responsibility of showing the result to the **Display**.

The display should also keep track of the last result shown.

Aggregation Task

In a classroom, multiple students are solving math problems. However, instead of each student having their own calculator, the teacher provides one **shared calculator** that all students can use.

Each student can:

- Use the calculator to perform addition or multiplication.
- Request the result of an operation by calling the calculator.

The **calculator is not owned by any student**. it is created and managed externally (by the teacher or system), and students just refer to it.

Composition

cpp

```
#include <iostream>

class Display {
    double lastValue;
public:
    Display() {
        lastValue = 0.0;
    }

    void show(double value) {
        lastValue = value;
        std::cout << "Result: " << lastValue << "\n";
    }

    double getLastValue() const {
        return lastValue;
    }
};
```

```
class Calculator {
    Display screen; // COMPOSITION: screen is a part of Calculator
public:
    Calculator() {}

    void add(double a, double b) {
        double result = a + b;
        screen.show(result);
    }

    void multiply(double a, double b) {
        double result = a * b;
        screen.show(result);
    }

    void showPreviousResult() const {
        std::cout << "Previous result: " << screen.getLastValue() << "\n";
    }
};

int main() {
    Calculator calc;

    calc.add(10, 5);           // Output: 15
    calc.multiply(3, 7);       // Output: 21
    calc.showPreviousResult(); // Output: 21

    return 0;
}
```

Aggregation

 Copy code

```
int main() {  
    Calculator myCalculator; // Created outside the stu  
  
    Student s1(&myCalculator);  
    Student s2(&myCalculator);  
  
    std::cout << "--- Student 1 ---\n";  
    s1.solveAddition(8, 3);  
    s1.solveMultiplication(4, 5);  
  
    std::cout << "\n--- Student 2 ---\n";  
    s2.solveAddition(10, 2);  
    s2.solveMultiplication(6, 7);  
  
    return 0;  
}
```

```
#include <iostream>  
  
class Calculator {  
public:  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    double multiply(double a, double b) {  
        return a * b;  
    }  
};  
  
class Student {  
    Calculator* calc; // Aggregation: uses an external calculator  
public:  
    Student(Calculator* c) {  
        calc = c;  
    }  
  
    void solveAddition(double a, double b) {  
        if (calc) {  
            std::cout << "Addition result: " << calc->add(a, b) << "\n";  
        }  
    }  
  
    void solveMultiplication(double a, double b) {  
        if (calc) {  
            std::cout << "Multiplication result: " << calc->multiply(a, b) << "\n";  
        }  
    }  
};
```